Distributed Machine Learning - Server-based e Peer-to-Peer

Programação Paralela – BCC447

Matheus Peixoto Ribeiro Vieira

27 de janeiro de 2026

Gradiente e otimização

Considere uma rede (camada linear):

$$Z = WX + b$$

Objetivo: encontrar os melhores parâmetros W e b minimizando a função de perda.

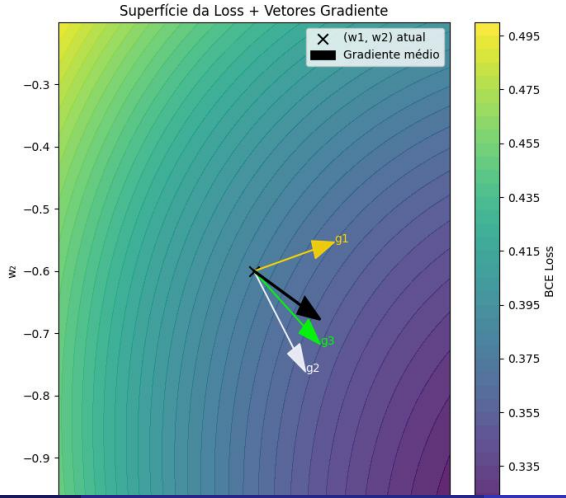
Atualização por descida do gradiente:

$$\theta_{t+1} = \theta_t - \alpha\, \hat{g}_t$$

- $\hat{g}_t$: gradiente estimado (mini-batch)
- α: learning rate

SGD é ruidoso

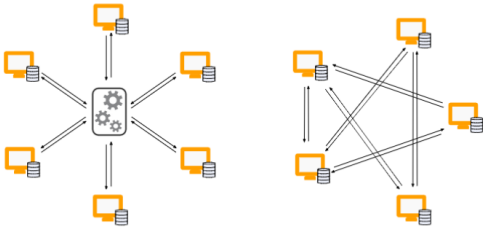
Cada mini-batch pode induzir um gradiente diferente, levando a direções ligeiramente distintas.



Acelerando o processo: Distributed SGD (D-SGD)

Fases do D-SGD:

1. Computação local
2. Comunicação
3. Atualização global



(a) Server based (b) Peer-to-peer

Fig. 3. Server-based and peer-to-peer system architectures with six and five nodes, respectively.

Servidor central: visão geral

Gradientes locais são enviados a um servidor/aggregator central, que combina as atualizações.

Algorithm 1 `FederatedAveraging`. The K clients are indexed by k; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:
initialize w_0
for each round $t = 1, 2, \dots$ **do**
  $m \leftarrow \max(C \cdot K, 1)$
  $S_t \leftarrow$ (random set of m clients)
  for each client $k \in S_t$ **in parallel do**
    $w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$
  $m_t \leftarrow \sum_{k \in S_t} n_k$
  $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$

ClientUpdate(k, w): *// Run on client k*
$\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)
for each local epoch i from 1 to E **do**
  for batch $b \in \mathcal{B}$ **do**
    $w \leftarrow w - \eta \nabla \ell(w; b)$
return w to server

Problema: média convencional sob ataque bizantino

Na agregação por média, um nó bizantino pode enviar um gradiente que:

- tem **alta magnitude**
- aponta na **direção oposta**

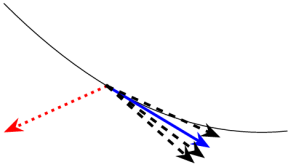


Figure 1: The gradient estimates computed by correct workers (black dashed arrows) are distributed around the actual gradient (solid arrow) of the cost function (thin black curve). A Byzantine worker can propose an arbitrary vector (red dotted arrow).

Intuição: gradientes honestos vs. maliciosos

Gradientes honestos:

- são **ruidosos** por causa dos mini-batches
- tendem a ser **próximos** entre si

Gradientes maliciosos:

- desviam a atualização global para **não convergir**
- exploram a sensibilidade da média a aoutliers

Algoritmo de Krum

Krum busca um agregador robusto a nós bizantinos:

- (i) Não ser enganado por outliers
- (ii) Não descartar gradientes honestos em excesso

Objetivo: gerar um gradiente agregado g que

- aponte para uma direção correta
- não exploda em magnitude

$$\langle \mathbb{E}[F], g \rangle \geq (1 - \sin \alpha)|g|^2$$

Krum: ideia central

- Não combina todos os vetores em um só.
- Seleciona um gradiente que pareça certo e com uma boa vizinhança

Krum: algoritmo

Dados vetores de gradientes $V_1, \ldots, V_n$:

1. Calcule as distâncias para todos os pares:

$$\|V_i - V_j\|^2 \quad \forall j \neq i$$

2. Para cada i, selecione os $n - f - 2$ vizinhos mais próximos e chame de N_i
3. Score:

$$s(i) = \sum_{j \in N_i} \|V_i - V_j\|^2$$

4. Escolha:

$$\arg\min_i \; s(i)$$

Custo: $O(n^2 \cdot d)$ (nós $\times$ dimensão do gradiente).

Multi-Krum

Escolher um único gradiente pode desacelerar o treino.

Multi-Krum:

- Seleciona até m vetores com menores scores
- Faz a média desses m vetores

Casos extremos:

- $m = 1 \Rightarrow$ Krum
- $m = n \Rightarrow$ média convencional

O que o Krum resolve (e o que não resolve)

Resolve bem:

- outliers aleatórios
- ataques de grande magnitude que puxam a média

Não verifica:

- se o gradiente é bom para o treinamento
- apenas **consistência geométrica** (proximidade/distâncias)

Ataque ao Krum: intuição

Objetivo do atacante: Modificar a direção do gradiente sem parecer um outlier.

Estratégia:

- criar um **cluster malicioso compacto** (gradientes maliciosos próximos entre si)
- ficar **próximo de alguns gradientes honestos** (explorando o ruído do SGD)
- apontar para uma direção **ruim** mas **plausível**

Por que funciona

O Krum procura o vetor com melhor vizinhança. Se o atacante garante uma boa vizinhança, então:

- o vetor malicioso **não é extremo**
- fica **dentro da nuvem** de gradientes
- o cluster malicioso é **muito consistente** $\Rightarrow$ score pequeno
- o Krum pode selecionar o vetor malicioso como "mais confiável"

Peer-to-Peer (P2P): visão geral

A comunicação é definida por um **grafo direcionado**. Cada nó mantém e atualiza um modelo local interagindo com seus vizinhos.

Problemas principais:

- falta de confiança
- nós bizantinos
- privacidade (gradientes podem vazar informação)

SPDL: o que promete

SPDL busca combinar:

- **descentralização**
- **privacidade**
- **tolerância a bizantinos**
- **auditabilidade** (blockchain)

SPDL: inicialização

- Cada nó gera chaves PK (pública) e SK (privada) e um id de 256 bits
- Cria um bloco gênesis (nó inicial da blockchain)
- O nó só participa se tiver registro (pk, id, IP, etc.)

Parâmetros globais (ex.):

- reputação inicial (popularidade) igual para todos
- learning rate, número de rodadas T
- variância do ruído gaussiano

SPDL: decisão do líder

Algorithm 1: Leader Election

1 (h_i, π_i) = VRF(sk_i, $seed$)
2 $L_i = (h_i, \pi_i, id_i)$
3 Broadcast (h_i, π_i) to the network and receive (h, π) form peers
4 **while** *TRUE* **do**
5 **if** *receive* (h_j, π_j) *from* id_j *&&* *VerifyVRF*$(pk, h_j, \pi_j, seed)$*=1* **then**
6 add (h_j, π_j, id_j) to L_i
7 **if** `Time()` $> start + \delta_1$ **then**
8 select the largest h_{max} from L_i and obtain id_{max}
  Output: id_{max}

Intuição de segurança: não é possível forjar h e π para outros nós.

SPDL: cálculo do gradiente com privacidade

Cada nó:

- calcula SGD localmente
- adiciona **ruído gaussiano** ao gradiente
- transmite o gradiente ruidoso e recebe o dos outros

SPDL: consenso BFT em quatro fases

Em vez de Proof-of-Work / Proof-of-Stake, SPDL usa um consenso **BFT**.

Fases:

1. **PRE-PREPARE**
2. **PREPARE**
3. **COMMIT**
4. **DECIDE**

PRE-PREPARE

▷ PRE−PREPARE

if *role is leader* **then**

$\Delta^{(t)} = \mathcal{K}(g_1^{(t)}, g_2^{(t)}, \cdots, g_n^{(t)})$

$B_k^{(t)} \leftarrow \texttt{MSGB}(\Delta^{(t)})$

broadcast $\langle \textbf{PRE-PREPARE}, id, B_k^{(t)}, h \rangle_{\hat{\sigma}}$

PREPARE

7 ▷ PREPARE
8 **if** *role is follower* **then**
9 compute $\tilde{\Delta}^{(t)} = \mathcal{K}(g_1^{(t)}, g_2^{(t)}, \cdots, g_n^{(t)})$
10 **while** *receive* $\langle \texttt{PRE-PREPARE}, id, B_k^{(t)}, h \rangle_{\hat{\sigma}}$ **do**
11 **if** σ *and* $B_k^{(t)}$ *are valid and* $\tilde{\Delta}^{(t)} \approx B_k^{(t)}.\Delta^{(t)}$ **then**
12 broadcast $\langle \texttt{PREPARE}, id, h, vote \rangle_{\hat{\sigma}}$

COMMIT

13 ▷ COMMIT
14 **while** *receive* $2f + 1$ $\langle \texttt{PREPARE}, id, h, vote \rangle_{\hat{\sigma}}$ **do**
15 broadcast $\langle \texttt{COMMIT}, id, vote \rangle_{\hat{\sigma}}$

DECIDE

16 ▷ DECIDE
17 **while** *receive* $2f + 1$ $\langle \texttt{COMMIT}, id, h, vote \rangle_{\hat{\sigma}}$ **do**
18 $\quad$ $\texttt{Append}(BC_k^{(t)}, B_k^{(t)})$
19 $\quad$ $x_i^{(t+1)} = x_i^{(t)} - \gamma \Delta^{(t)}$
20 $\quad$ Update reputation

Por que blockchain ajuda

Vantagens:

- só entra no histórico o que recebeu **quórum** e foi verificado
- histórico imutável
- permite **auditar** e **punir** nós suspeitos (reduzindo reputação)

Efeito do papel de líder:

- menos dependência dos outros para avançar
- menor risco de ficar para trás
- maior controle do ritmo de treino

Referências

- Byzantine Machine Learning: A Primer - Guerraoui et al., 2024
- Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent - Blanchard et al., 2017
- Local Model Poisoning Attacks to Byzantine-Robust Federated Learning - Fang et al., 2020
- SPDL: Blockchain-secured and Privacy-preserving Decentralized Learning - Xu et al., 2022

Obrigado